Test-Driven Development Documentation

Myngly Ads Unified API

Version: 1.6.0 **Date:** 2026-01-28 **Coverage:** 85.48% (1,008 tests)

Table of Contents

1. Overview
2. Test Architecture
3. Coverage Summary
4. Testing Patterns
5. Domain Test Files
6. Mocking Strategies
7. Running Tests
8. Writing New Tests
9. CI/CD Integration

---

1. Overview

TDD Methodology

This project follows Test-Driven Development principles:

1. **RED**: Write a failing test first
2. **GREEN**: Write minimal code to pass the test
3. **REFACTOR**: Improve code while keeping tests green

Testing Stack

Tool	Purpose	Version
Vitest	Test runner	2.x
Supertest	HTTP assertions	7.x
@vitest/coverage-v8	Coverage reporting	2.x

Coverage Targets

Metric	Target	Achieved
Statements	80%	85.48%
Branches	80%	90.69%
Functions	80%	84.48%
Lines	80%	85.48%

---

2. Test Architecture

Directory Structure

```
src/
├── domains/
│   ├── identity/
│   │   ├── services/
│   │   │   └── __tests__/
│   │   │       ├── auth.service.test.ts
│   │   │       ├── admin-auth.service.test.ts
│   │   │       └── token.service.test.ts
│   │   ├── controllers/
│   │   │   └── __tests__/
│   │   │       ├── auth.controller.test.ts
│   │   │       └── admin-auth.controller.test.ts
│   │   ├── repositories/
│   │   │   └── __tests__/
│   │   │       ├── user.repository.test.ts
│   │   │       └── refresh-token.repository.test.ts
│   │   └── validators/
│   │       └── __tests__/
│   │           └── auth.validator.test.ts
│   ├── campaign/
│   │   └── ... (same structure)
│   ├── user/
│   ├── payment/
│   ├── notification/
│   ├── analytics/
│   ├── administration/
│   └── platform/
├── infrastructure/
│   ├── http/
│   │   └── __tests__/
│   │       └── response.helper.test.ts
│   └── middleware/
│       └── __tests__/
│           ├── csrf.middleware.test.ts
│           └── rate-limit.middleware.test.ts
└── tests/
    ├── e2e/
    │   ├── security.test.ts
    │   ├── advertiser-journey.test.ts
    │   ├── payment-journey.test.ts
    │   └── admin-journey.test.ts
    ├── fixtures/
    │   ├── users.fixture.ts
    │   ├── campaigns.fixture.ts
    │   └── payments.fixture.ts
    └── helpers/
        ├── test-server.ts
        ├── mock-repositories.ts
```

```
        ├── mock-services.ts
        └── e2e-helpers.ts
```

Test Types

Type	Location	Purpose	Count
Unit	`src/**/__tests__/*.test.ts`	Service/utility logic	~800
Integration	`src/**/__tests__/*.integration.test.ts`	API endpoints	~100
E2E	`tests/e2e/*.test.ts`	Full user flows	~100

---

3. Coverage Summary

By Domain

Domain	Files	Statements	Branches	Functions
Identity	18	95.2%	92.1%	94.8%
Campaign	14	91.3%	88.5%	90.2%
User	12	89.7%	86.4%	88.1%
Payment	16	99.8%	98.2%	99.5%
Notification	10	87.4%	85.1%	86.9%
Analytics	6	88.2%	84.7%	87.3%
Administration	8	86.5%	83.9%	85.8%
Platform	8	92.1%	89.3%	91.4%
Infrastructure	12	86.1%	85.7%	100%

By Layer

Layer	Coverage
Controllers	100%
Services	98.5%
Repositories	95.2%
Validators	100%
Middleware	98.3%

---

4. Testing Patterns

4.1 Unit Test Pattern

```typescript
// src/domains/identity/services/__tests__/auth.service.test.ts
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { AuthService } from '../auth.service';
import type { IUserRepository } from '../../interfaces';

describe('AuthService', () => {
  let authService: AuthService;
  let mockUserRepo: IUserRepository;
  let mockTokenService: ITokenService;
  let mockPasswordService: IPasswordService;

  beforeEach(() => {
    // Create mocks
    mockUserRepo = {
      findByEmail: vi.fn(),
      findById: vi.fn(),
      create: vi.fn(),
      update: vi.fn(),
    };

    mockTokenService = {
      generateAccessToken: vi.fn(),
      generateRefreshToken: vi.fn(),
      verifyAccessToken: vi.fn(),
    };

    mockPasswordService = {
      hash: vi.fn(),
      verify: vi.fn(),
    };

    // Inject mocks
    authService = new AuthService(
      mockUserRepo,
      mockTokenService,
      mockPasswordService
    );
  });

  describe('login', () => {
    it('should return tokens when credentials are valid', async () => {
      // Arrange
      const email = 'test@example.com';
      const password = 'ValidPass123!';
      const user = { id: '1', email, passwordHash: 'hashed' };

      mockUserRepo.findByEmail.mockResolvedValue(user);
      mockPasswordService.verify.mockResolvedValue(true);
      mockTokenService.generateAccessToken.mockReturnValue('access-token');
      mockTokenService.generateRefreshToken.mockResolvedValue('refresh-token');
```

```
      // Act
      const result = await authService.login(email, password);

      // Assert
      expect(result.accessToken).toBe('access-token');
      expect(result.refreshToken).toBe('refresh-token');
      expect(mockUserRepo.findByEmail).toHaveBeenCalledWith(email);
    });

    it('should throw UnauthorizedError when user not found', async () => {
      // Arrange
      mockUserRepo.findByEmail.mockResolvedValue(null);

      // Act & Assert
      await expect(authService.login('unknown@example.com', 'pass'))
        .rejects.toThrow('Invalid email or password');
    });

    it('should increment failed attempts on wrong password', async () => {
      // Arrange
      const user = { id: '1', email: 'test@example.com', failedLoginAttempts: 0 };
      mockUserRepo.findByEmail.mockResolvedValue(user);
      mockPasswordService.verify.mockResolvedValue(false);

      // Act & Assert
      await expect(authService.login(user.email, 'wrong'))
        .rejects.toThrow('Invalid email or password');
      expect(mockUserRepo.update).toHaveBeenCalledWith('1', {
        failedLoginAttempts: 1,
      });
    });
  });
});
```

4.2 Controller Test Pattern

```
// src/domains/identity/controllers/__tests__/auth.controller.test.ts
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { AuthController } from '../auth.controller';
import { createMockRequest, createMockResponse } from '../../../../tests/helpers';

describe('AuthController', () => {
  let controller: AuthController;
  let mockAuthService: IAuthService;
  let mockReq: MockRequest;
  let mockRes: MockResponse;

  beforeEach(() => {
    mockAuthService = {
      login: vi.fn(),
      register: vi.fn(),
```

```
      logout: vi.fn(),
      refreshToken: vi.fn(),
    };

    controller = new AuthController(mockAuthService);
    mockReq = createMockRequest();
    mockRes = createMockResponse();
  });

  describe('login', () => {
    it('should set cookies and return user on successful login', async () => {
      // Arrange
      mockReq.body = { email: 'test@example.com', password: 'Pass123!' };
      mockAuthService.login.mockResolvedValue({
        user: { id: '1', email: 'test@example.com' },
        accessToken: 'access-token',
        refreshToken: 'refresh-token',
      });

      // Act
      await controller.login(mockReq, mockRes);

      // Assert
      expect(mockRes.cookie).toHaveBeenCalledWith(
        'access_token',
        'access-token',
        expect.objectContaining({ httpOnly: true })
      );
      expect(mockRes.json).toHaveBeenCalledWith(
        expect.objectContaining({ success: true })
      );
    });

    it('should return 401 on invalid credentials', async () => {
      // Arrange
      mockReq.body = { email: 'test@example.com', password: 'wrong' };
      mockAuthService.login.mockRejectedValue(
        new UnauthorizedError('Invalid email or password')
      );

      // Act
      await controller.login(mockReq, mockRes);

      // Assert
      expect(mockRes.status).toHaveBeenCalledWith(401);
    });
  });
});
```

4.3 Repository Test Pattern

```typescript
// src/domains/identity/repositories/__tests__/user.repository.test.ts
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { UserRepository } from '../user.repository';
import { prismaMock } from '../../../../../tests/helpers/prisma-mock';

describe('UserRepository', () => {
  let repository: UserRepository;

  beforeEach(() => {
    repository = new UserRepository(prismaMock);
  });

  describe('findByEmail', () => {
    it('should return user when found', async () => {
      // Arrange
      const mockUser = {
        id: 1,
        uuid: 'uuid-123',
        email: 'test@example.com',
        passwordHash: 'hashed',
      };
      prismaMock.user.findUnique.mockResolvedValue(mockUser);

      // Act
      const result = await repository.findByEmail('test@example.com');

      // Assert
      expect(result).toEqual(mockUser);
      expect(prismaMock.user.findUnique).toHaveBeenCalledWith({
        where: { email: 'test@example.com' },
      });
    });

    it('should return null when not found', async () => {
      // Arrange
      prismaMock.user.findUnique.mockResolvedValue(null);

      // Act
      const result = await repository.findByEmail('unknown@example.com');

      // Assert
      expect(result).toBeNull();
    });
  });
});
```

4.4 E2E Test Pattern

```typescript
// tests/e2e/advertiser-journey.test.ts
import { describe, it, expect, beforeAll, afterAll } from 'vitest';
```

```typescript
import request from 'supertest';
import { app } from '../../src/app';
import { getCsrfToken, authRequest } from '../helpers/e2e-helpers';

describe('Advertiser Journey', () => {
  let csrfToken: string;
  let accessToken: string;

  beforeAll(async () => {
    csrfToken = await getCsrfToken(app);
  });

  describe('Registration Flow', () => {
    it('should register a new user', async () => {
      const response = await request(app)
        .post('/api/v1/auth/register')
        .set('X-CSRF-Token', csrfToken)
        .send({
          email: 'newuser@example.com',
          password: 'ValidPass123!',
          name: 'Test User',
        });

      expect(response.status).toBe(201);
      expect(response.body.success).toBe(true);
      expect(response.body.data.user.email).toBe('newuser@example.com');
    });

    it('should reject duplicate email', async () => {
      const response = await request(app)
        .post('/api/v1/auth/register')
        .set('X-CSRF-Token', csrfToken)
        .send({
          email: 'newuser@example.com',
          password: 'ValidPass123!',
          name: 'Duplicate User',
        });

      expect(response.status).toBe(409);
      expect(response.body.code).toBe('CONFLICT');
    });
  });

  describe('Login Flow', () => {
    it('should login and receive cookies', async () => {
      const response = await request(app)
        .post('/api/v1/auth/login')
        .set('X-CSRF-Token', csrfToken)
        .send({
          email: 'newuser@example.com',
          password: 'ValidPass123!',
        });
```

```
      expect(response.status).toBe(200);
      expect(response.headers['set-cookie']).toBeDefined();

      // Extract token for subsequent requests
      accessToken = response.headers['set-cookie']
        .find((c: string) => c.startsWith('access_token'))
        ?.split(';')[0]
        ?.split('=')[1];
    });
  });

  describe('Campaign CRUD', () => {
    let campaignId: string;

    it('should create a campaign', async () => {
      const response = await authRequest(app, accessToken, csrfToken)
        .post('/api/v1/campaigns')
        .send({
          name: 'Test Campaign',
          promotionType: 'USER',
          adFormat: 'BANNER',
          budget: 100,
          startDate: '2026-02-01',
          endDate: '2026-02-28',
        });

      expect(response.status).toBe(201);
      campaignId = response.body.data.id;
    });

    it('should list user campaigns', async () => {
      const response = await authRequest(app, accessToken, csrfToken)
        .get('/api/v1/campaigns');

      expect(response.status).toBe(200);
      expect(response.body.data.length).toBeGreaterThan(0);
    });

    it('should update campaign', async () => {
      const response = await authRequest(app, accessToken, csrfToken)
        .put(`/api/v1/campaigns/${campaignId}`)
        .send({ name: 'Updated Campaign' });

      expect(response.status).toBe(200);
      expect(response.body.data.name).toBe('Updated Campaign');
    });
  });
});
```

5. Domain Test Files

Identity Domain (119 tests)

File	Tests	Coverage
`auth.service.test.ts`	32	98%
`admin-auth.service.test.ts`	28	96%
`token.service.test.ts`	24	100%
`two-factor.service.test.ts`	18	95%
`auth.validator.test.ts`	12	100%
`admin-auth.validator.test.ts`	5	100%

Campaign Domain (72 tests)

File	Tests	Coverage
`campaign.service.test.ts`	16	95%
`campaign-review.service.test.ts`	16	94%
`campaign.validator.test.ts`	22	100%
`admin-campaign.validator.test.ts`	18	100%

User Domain (106 tests)

File	Tests	Coverage
`user-profile.service.test.ts`	24	96%
`admin-user-management.service.test.ts`	28	94%
`user.validator.test.ts`	18	100%
`admin-user.validator.test.ts`	16	100%
`notification-setting.service.test.ts`	20	95%

Payment Domain (109 tests)

File	Tests	Coverage
`stripe-customer.service.test.ts`	18	100%
`stripe-payment.service.test.ts`	28	100%
`stripe-invoice.service.test.ts`	21	100%
`stripe-refund.service.test.ts`	14	100%

stripe-webhook.service.test.ts	22	100%
stripe-admin.service.test.ts	24	100%
payment-method.service.test.ts	12	100%
payment.validator.test.ts	8	100%

Notification Domain (47 tests)

File	Tests	Coverage
email.service.test.ts	15	92%
email-template-account.service.test.ts	14	100%
email-template-campaign.service.test.ts	18	100%

Analytics Domain (21 tests)

File	Tests	Coverage
dashboard.service.test.ts	12	94%
report.service.test.ts	9	90%

Administration Domain (24 tests)

File	Tests	Coverage
settings.service.test.ts	12	92%
audit.service.test.ts	12	94%

Platform Domain (16 tests)

File	Tests	Coverage
ad-format.service.test.ts	8	100%
boost-type.service.test.ts	8	100%

Infrastructure (92 tests)

File	Tests	Coverage
response.helper.test.ts	18	100%
error.handler.test.ts	12	76%
csrf.middleware.test.ts	12	100%
rate-limit.middleware.test.ts	6	100%
validation.middleware.test.ts	8	100%

`crypto.service.test.ts`	12	100%
`password.service.test.ts`	8	100%
`jwt.service.test.ts`	16	100%

E2E Tests (71 tests)

File	Tests	Purpose
`security.test.ts`	10	CSRF, Helmet, rate limiting
`advertiser-journey.test.ts`	15	Registration, login, campaigns
`payment-journey.test.ts`	13	Payment methods, Stripe
`admin-journey.test.ts`	24	Admin auth, 2FA, management
`auth.e2e.test.ts`	9	Full auth flow

---

6. Mocking Strategies

6.1 Prisma Client Mock

```
// tests/helpers/prisma-mock.ts
import { PrismaClient } from '@prisma/client';
import { mockDeep, mockReset, DeepMockProxy } from 'vitest-mock-extended';

export const prismaMock = mockDeep<PrismaClient>();

beforeEach(() => {
  mockReset(prismaMock);
});
```

6.2 Stripe SDK Mock

```
// tests/helpers/stripe-mock.ts
import Stripe from 'stripe';
import { vi } from 'vitest';

export const createStripeMock = () => ({
  customers: {
    create: vi.fn(),
    retrieve: vi.fn(),
    update: vi.fn(),
    del: vi.fn(),
  },
  paymentIntents: {
    create: vi.fn(),
    confirm: vi.fn(),
```

```
    retrieve: vi.fn(),
    cancel: vi.fn(),
  },
  invoices: {
    create: vi.fn(),
    retrieve: vi.fn(),
    list: vi.fn(),
    pay: vi.fn(),
    finalizeInvoice: vi.fn(),
    voidInvoice: vi.fn(),
  },
  refunds: {
    create: vi.fn(),
    retrieve: vi.fn(),
    list: vi.fn(),
  },
  webhooks: {
    constructEvent: vi.fn(),
  },
  balance: {
    retrieve: vi.fn(),
  },
});
```

6.3 Request/Response Mocks

```
// tests/helpers/express-mock.ts
import { vi } from 'vitest';
import type { Request, Response } from 'express';

export const createMockRequest = (overrides = {}): Partial<Request> => ({
  body: {},
  params: {},
  query: {},
  headers: {},
  cookies: {},
  user: undefined,
  ...overrides,
});

export const createMockResponse = (): Partial<Response> => {
  const res: Partial<Response> = {};
  res.status = vi.fn().mockReturnValue(res);
  res.json = vi.fn().mockReturnValue(res);
  res.cookie = vi.fn().mockReturnValue(res);
  res.clearCookie = vi.fn().mockReturnValue(res);
  res.send = vi.fn().mockReturnValue(res);
  return res;
};
```

6.4 Service Mock Factory

```
// tests/helpers/mock-services.ts
import { vi } from 'vitest';

export const createMockAuthService = () => ({
  login: vi.fn(),
  register: vi.fn(),
  logout: vi.fn(),
  refreshToken: vi.fn(),
  verifyEmail: vi.fn(),
  forgotPassword: vi.fn(),
  resetPassword: vi.fn(),
});

export const createMockCampaignService = () => ({
  findAll: vi.fn(),
  findById: vi.fn(),
  create: vi.fn(),
  update: vi.fn(),
  delete: vi.fn(),
  updateStatus: vi.fn(),
  duplicate: vi.fn(),
});

// ... more service mocks
```

---

7. Running Tests

Commands

```
# Run all tests
npm test

# Run with coverage
npm run test:coverage

# Run specific file
npm test -- auth.service.test.ts

# Run specific domain
npm test -- src/domains/identity

# Run E2E only
npm test -- tests/e2e

# Watch mode
npm run test:watch
```

```
# Update snapshots
npm test -- -u
```

Configuration

```
// vitest.config.ts
import { defineConfig } from 'vitest/config';
import path from 'path';

export default defineConfig({
  test: {
    globals: true,
    environment: 'node',
    include: ['src/**/*.test.ts', 'tests/**/*.test.ts'],
    exclude: ['node_modules', 'dist'],
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      exclude: [
        'node_modules',
        'dist',
        '**/*.d.ts',
        '**/*.test.ts',
        '**/types/**',
        '**/interfaces/**',
      ],
      thresholds: {
        global: {
          statements: 80,
          branches: 80,
          functions: 80,
          lines: 80,
        },
      },
    },
    setupFiles: ['./tests/setup.ts'],
  },
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
});
```

---

8. Writing New Tests

Checklist for New Features

1. **Write failing test first** (RED)

- Define expected behavior
- Use descriptive test name: `should [behavior] when [condition]`

2. **Implement minimal code** (GREEN)
   - Make test pass with simplest solution
   - Don't over-engineer
3. **Refactor** (IMPROVE)
   - Clean up code
   - Ensure tests still pass

Test Naming Convention

```
describe('ServiceName', () => {
  describe('methodName', () => {
    it('should [expected behavior] when [condition]', () => {
      // ...
    });
  });
});

// Examples:
it('should return 401 when token is expired')
it('should create campaign with targeting when all fields provided')
it('should lock account when 5 failed login attempts occur')
it('should send welcome email when user registers')
```

Coverage Requirements

- **New services**: 90%+ coverage required
- **New controllers**: 85%+ coverage required
- **New repositories**: 80%+ coverage required
- **Bug fixes**: Include regression test

Pull Request Checklist

- [ ] All new code has tests
- [ ] Tests follow naming convention
- [ ] Coverage threshold maintained (80%+)
- [ ] No skipped tests (`it.skip`)
- [ ] No focused tests (`it.only`)
- [ ] E2E test for critical paths

---

9. CI/CD Integration

GitHub Actions Workflow

```
# .github/workflows/test.yml
name: Tests
```

```yaml
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: myngly_test
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=3

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Generate Prisma client
        run: npm run db:generate

      - name: Run migrations
        run: npx prisma migrate deploy
        env:
          DATABASE_URL: mysql://root:root@localhost:3306/myngly_test

      - name: Type check
        run: npm run typecheck

      - name: Lint
        run: npm run lint

      - name: Run tests
```

```
        run: npm run test:coverage
        env:
          NODE_ENV: test
          DATABASE_URL: mysql://root:root@localhost:3306/myngly_test
          JWT_ACCESS_SECRET: test-access-secret
          JWT_REFRESH_SECRET: test-refresh-secret

      - name: Upload coverage
        uses: codecov/codecov-action@v3
        with:
          files: ./coverage/coverage-final.json
          fail_ci_if_error: true
```

Pre-commit Hook

```
# .husky/pre-commit
#!/bin/sh
npm run lint-staged
npm test -- --run --passWithNoTests
```

Coverage Badge

Add to README.md:

```
![Coverage](https://img.shields.io/badge/coverage-85.48%25-brightgreen)
```

Appendix A: Test Fixtures

User Fixtures

```
// tests/fixtures/users.fixture.ts
export const testUser = {
  id: 1,
  uuid: 'user-uuid-123',
  email: 'test@example.com',
  name: 'Test User',
  passwordHash: '$2a$12$...',
  emailVerified: true,
  status: 'ACTIVE',
  createdAt: new Date('2026-01-01'),
};

export const testAdminUser = {
  id: 1,
  uuid: 'admin-uuid-123',
  email: 'admin@myngly.com',
  name: 'Admin User',
  role: 'SUPER_ADMIN',
```

```
  permissions: ['*'],
  twoFactorEnabled: true,
};
```

Campaign Fixtures

```
// tests/fixtures/campaigns.fixture.ts
export const testCampaign = {
  id: 1,
  uuid: 'campaign-uuid-123',
  userId: 1,
  name: 'Test Campaign',
  status: 'DRAFT',
  promotionType: 'USER',
  adFormat: 'BANNER',
  budget: 100.00,
  spent: 0,
  impressions: 0,
  clicks: 0,
  startDate: new Date('2026-02-01'),
  endDate: new Date('2026-02-28'),
};
```

Appendix B: Troubleshooting

Common Issues

Tests timeout:

```
# Increase timeout in vitest.config.ts
test: {
  testTimeout: 30000,
}
```

Prisma mock not resetting:

```
// Add to beforeEach
import { mockReset } from 'vitest-mock-extended';
beforeEach(() => {
  mockReset(prismaMock);
});
```

Stripe mock not working:

```
// Mock the entire module
vi.mock('stripe', () => ({
  default: vi.fn(() => createStripeMock()),
}));
```

CSRF token missing in E2E:

```
// Get token before making requests
const csrfToken = await getCsrfToken(app);
request(app)
  .post('/api/v1/...')
  .set('X-CSRF-Token', csrfToken)
```

Generated: 2026-01-28 Test Framework: Vitest 2.x Coverage: 85.48% (1,008 tests)